DA3408 Assignment 3: Spark vs. Ray — The Data Engineering Duel

M M Pruthvi Raj Roll no.: **DA24B016** Code: https://github.com/Pruthvi016-sys/aiops-assignment-3

1 Setup and pipeline

Data. NYC TLC Yellow Taxi trips, Jan 2021 – Aug 2024: 44 Parquet files, 2.1 GB, 135,258,811 raw rows, plus `taxi_zone_lookup.csv`. The files have two schema variants (`passenger_count` is double before Feb 2023 and int64 after; IDs are int64 vs int32), so both pipelines cast every column to one canonical type before combining files.

Hardware and clusters. One HP 15s laptop (12 logical cores, 14 GB RAM, Ubuntu, Java 17, Python 3.11, Spark 3.5.9, Ray 2.59.0). I configured both clusters manually. **Spark:** standalone master plus two workers (4 cores, 3 GB each; Fig. 4a,b). **Ray:** a head node plus one worker node (4 CPUs each; Fig. 4c). *Note on "two workers":* in Ray the head node also executes tasks, so the two Ray nodes are the two task-executing nodes and play the role of Spark's two workers (the Spark master schedules but does not execute tasks). Both frameworks therefore have the same compute budget, 8 cores/CPUs. A strict reading of "two workers" would require a head with 0 CPUs plus two worker nodes; I did not run that configuration. **Both clusters share one physical machine** (CPU, RAM, disk), so this is a single-host simulation of a cluster and absolute numbers should be read in that light.

Pipeline parity. `spark_clean.py` and `ray_clean.py` import the same rules and the same Python function `trip_speed_mph` from `common.py`. Steps: drop nulls; cast timestamps to a canonical microsecond timestamp (session time zone UTC; duration is the whole-second difference of pickup and drop-off), keeping pickups in [2021-01-01, 2024-09-01); passengers 1–6; distance, fare and total in bounded positive ranges; tip in $[0, 500]$; duration 60 s–6 h; exact duplicate removal over the 11 columns; inner join with the zone table twice (pickup and drop-off); `pickup_hour`; `avg_speed_mph` via the UDF, kept in $(0, 80]$; write Parquet. Two points where the wording of the assignment is read loosely: the "heavy join" is against the 265-row zone table (broadcast in Spark), but it is applied twice to every one of the 123 M cleaned rows, as in the assignment's own example; and "average speed" is computed per trip together with a `pickup_hour` feature, with no per-hour aggregation. The 11-file benchmarks use 11 files spaced evenly over the 44 (first and last included, so both schema variants are covered).

Method. A harness (`benchmark.py`) runs each pipeline as a subprocess, takes wall-clock time from ingestion to export (output verification excluded), samples system-wide CPU% and used memory every second with `psutil` (equivalent to `top`), and fingerprints the output with an order-independent row hash (`compare_outputs.py`). "Memory Δ" is peak used memory minus the pre-run baseline.

2 Results

2.1 Parity

Files	Spark rows	Ray rows	Row hash (both)
2 (dedup on)	3,805,035	3,805,035	`5a139995bd24ac09`
11 (dedup on)	30,231,548	30,231,548	`afaacbedeb7b48da`
44 (dedup on)	123,047,199	123,047,199	`33ae82d94015b13a`
11 (dedup off)	30,231,550	30,231,550	`f51d7589a039edd2`

Table 1: Spark and Ray produce value-identical rows at every scale. The check compares canonicalised column values (same types, timestamps in microseconds) with an order-independent row hash and the row count; it does not compare Parquet file bytes. Dedup removes only 2 rows in the 11-file sample.

2.2 Execution time

Scenario (Python UDF)	Spark (s)	Ray (s)	Ray speed-up
11 files, dedup off, identical work (n=2 each)	80.7 ± 8.4	60.9 ± 0.7	**1.3**×
11 files, dedup on (n=4 each)	103.5 ± 6.3	42.5 ± 4.7	2.4×
44 files, dedup on (1 run each)	371.9	182.1	2.0×
2 files, dedup on (1 run each)	26.75	7.48	3.6×

Table 2: Wall-clock time, mean ± s.d. The dedup-off row runs the same work in both frameworks and is the primary like-for-like number. In the "dedup on" rows Ray uses the per-file strategy of Sec. 4 (a different, tuned algorithm and read path), so those speed-ups compare tuned Ray with default-configured Spark. The 44-file figures are single runs.

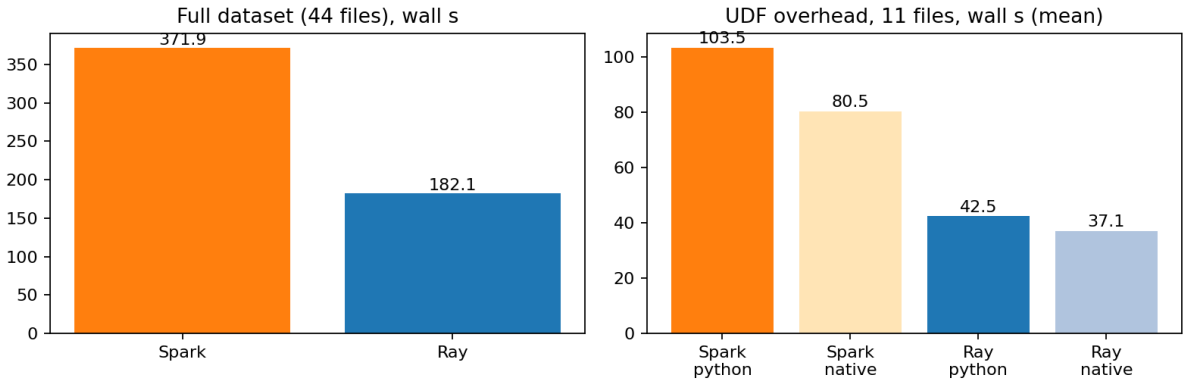


Figure 1: Left: full-dataset wall time (single runs). Right: Python-UDF vs. native-expression runs (11 files, mean).

2.3 Resource utilisation

Run	Mean CPU%	Peak CPU%	Peak mem Δ (GiB)	Peak used mem (GiB)
Spark, 44 files	90.5	100.0	8.09	10.75
Ray (per-file), 44 files	28.0	43.9	5.90	10.20
Spark, 11 files (run 1)	87.6	100.0	6.83	10.61
Ray (per-file), 11 files (run 1)	26.2	35.4	4.16	9.04
Ray global dedup (shuffle), 11 files	14.5	97.1	9.05	12.92

Table 3: System-wide CPU (12 logical cores) and memory from 1 s samples; they include both framework daemons and the OS. The pre-run baseline differed (Spark 2.7 GiB, Ray 4.3 GiB for the full runs), so peak *used* memory is the fairer figure: the two peak within about 0.6 GiB of each other.

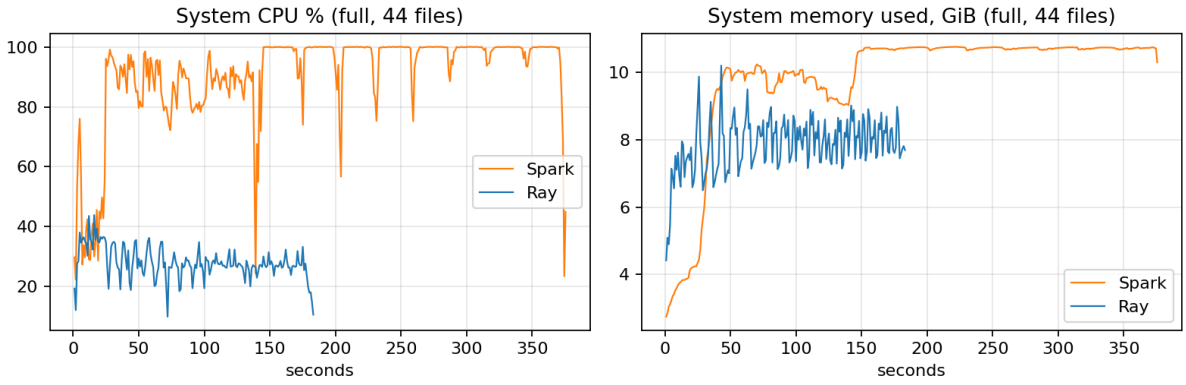


Figure 2: CPU and memory timelines of the full 44-file runs.

Spark kept the CPU near saturation, while Ray's per-file tasks used about a third of the machine (they are capped at 4 concurrent files by `num_cpus=2` per task, chosen to bound memory). I did not profile what Spark's CPU time was spent on (shuffle, serialisation, JVM garbage collection or the UDF), so I make no claim about the cause; high utilisation here should not be read as efficiency.

3 Python UDF

Both frameworks ran the same function (`python` mode) and an equivalent built-in expression (`native` mode) on 11 files.

	Python UDF (s)	Native (s)	Gap (s)	Gap (%)	Gap / s.d. of UDF runs
Spark (n=4 / n=2)	103.5 ± 6.3	80.5 ± 0.9	23.0	28.6	3.7
Ray (n=4 / n=3)	42.5 ± 4.7	37.1 ± 0.3	5.4	14.6	1.1

Table 4: Gap = Python-UDF time minus native time (percentage relative to native). Spark's gap is about 3.7 standard deviations of its Python-UDF runs, so it is a real effect. Ray's gap is only about 1.1 standard deviations: the four Ray Python runs ranged from 37.8 to 47.0 s and the fastest equals the native time, so with n=4 **Ray's UDF overhead cannot be distinguished from run-to-run noise**.

Interpretation. Spark runs the plan in the JVM; a Python UDF forces each row to be serialised to a Python worker process and the result back, a cost that grows with row count. Ray Data runs the whole stage in a Python worker on pandas/Arrow batches, so there is no JVM crossing; what remains is calling a Python function once per row. This is consistent with the data: Spark pays a measurable penalty and Ray's is within noise. I therefore claim the direction of the Python-native advantage, but not its size, and I do not quote a ratio between the two.

What this does not show. (i) The UDF was not timed in isolation: the table is a differential between a UDF run and a native run, not a measured UDF duration. (ii) In Ray the Python mode is a per-row loop and the native mode is a vectorised numpy expression, so the Ray gap mixes "Python loop vs. vectorised" with any framework effect, and in Spark the gap mixes the JVM crossing with the Python per-row cost. (iii) Ray's low CPU use (28%) can hide extra work, since there is idle capacity to absorb it. (iv) A vectorised `pandas_udf` variant for Spark was not tried; it would likely shrink Spark's gap and is the obvious next experiment. (v) The native runs round differently (Spark half-up, numpy half-even), which is why their hashes differ from the Python-UDF hashes and from each other; native mode was used for timing only, never for parity.

4 Ray deduplication: what failed and why

Spark's `dropDuplicates` is a global shuffle. The direct Ray equivalent did not scale on a 14 GB host with 1 GB object stores per node (these experiments used the same head-plus-one-worker Ray cluster):

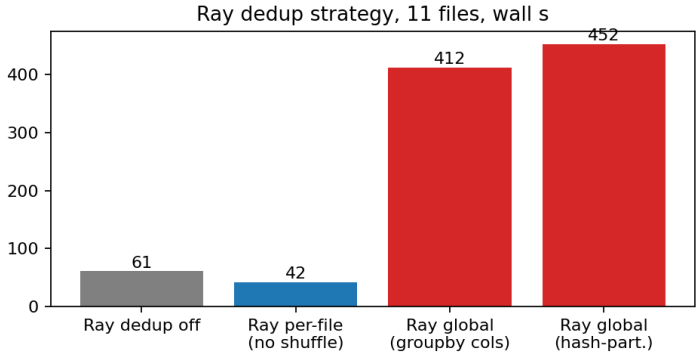


Figure 3: Ray on 11 files under different dedup strategies.

- **Global `groupby` dedup**: 411.8 s on 11 files (vs. 60.9 s with dedup off) with about 7.6 GiB spilled to disk.
- **Hash-partitioned dedup** (shuffle on a row hash, `drop_duplicates` per partition): correct on a self-test (`dedup_selftest.py`) and 452.1 s on 11 files. On all 44 files it stalled at `Shuffle 0/1` at about 4% CPU for more than 50 minutes, with a 40 GiB memory request and 6.9 GiB of queued blocks, then the GCS process was lost (log: `ray_full_FAILED_hash_attempt.log`).
- **Two-pass hash dedup**: after more than 10 minutes on 11 files it was still in the first aggregation pass, and the run then ended without a result. I did not capture the cause, so I make no claim that it was out-of-memory.
- **Per-file dedup (used for the Ray results)**: each monthly file is read, cleansed and de-duplicated inside one task; there is no shuffle. It is exact if no duplicate pair spans two files, which I verified rather than assumed: the output hash on all 44 files equals Spark's global-dedup hash (Table 1).

Confound. Per-file dedup is also a different ingestion path (a whole file read by PyArrow in one task, versus `ray.data.read_parquet` streaming 50,000-row batches). Ray with dedup *off* (60.9 s) is slower than Ray with per-file dedup (42.5 s), so Ray's "dedup on" speed-up comes from its read path, not from skipping work. For that reason the dedup-off row (1.3×) is the primary like-for-like number. Spark was not tuned beyond fixed executor memory/cores and 64 shuffle partitions, while Ray received a custom read path and a concurrency cap.

5 Conclusion

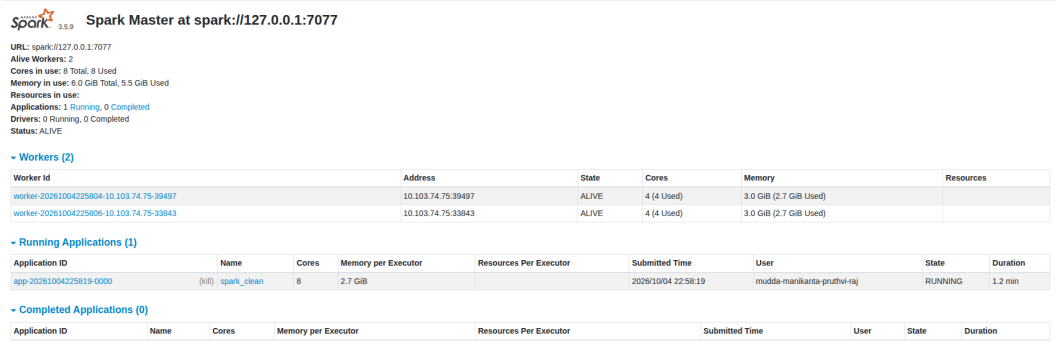
Winner on this benchmark: Ray, with moderate confidence. Like-for-like (dedup off, n=2 each) Ray is about 1.3× faster; on the full dataset the tuned Ray pipeline took 182 s vs. Spark's 372 s (2.0×), but that is one run each and compares a tuned Ray configuration with a default-configured Spark. Both clusters share one laptop, and Ray needed a custom dedup strategy to finish at all.

- **AI-first project (feature pipelines, embeddings, GPU/model stages, Python-native code):** choose Ray: Python-native batches, no JVM crossing for custom Python, and flexible task scheduling. Plan around shuffles.
- **BI-first project (SQL, joins, aggregations, exact global operations):** choose Spark: its global shuffle and `dropDuplicates` ran reliably at every scale, built-in expressions avoid the Python penalty, and the Spark UI gives better diagnostics.

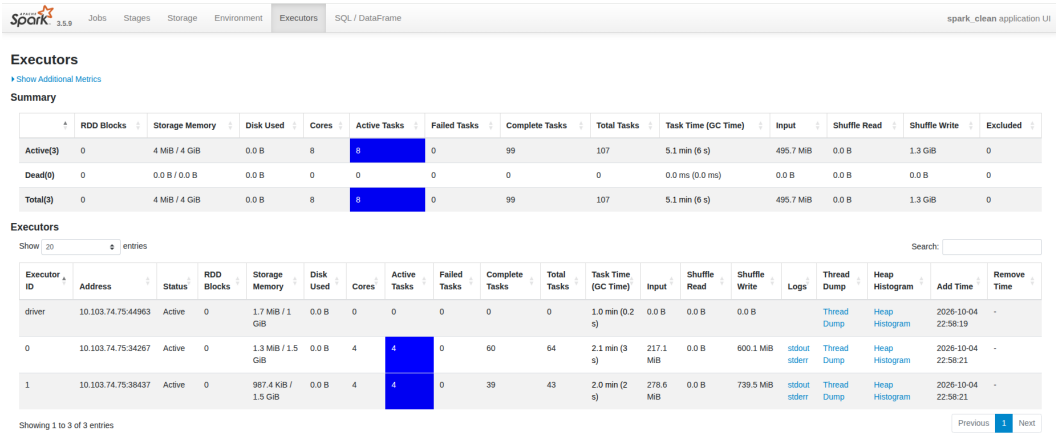
Performance Tuning Note and AI-use disclosure

An LLM (Claude) was used as an assistant, and the following came from its suggestions, which I then ran and checked: the translation of the PySpark logic into Ray Data/pandas; the canonical-type casting that reconciles the two schema variants; the `keep_default_na=False` fix (pandas turns the zone name "N/A" into NaN, Spark does not); the hash-partitioned, two-pass and per-file dedup variants and the dedup self-test; the `num_cpus=2` memory limit on the per-file task; the benchmark harness; and troubleshooting for Ray start-up (`RAY_AUTH_MODE=disabled`, and a shorter `--temp-dir` because of the 107-byte Unix-socket path limit). I configured and started both clusters on my own machine, and every number in this report was produced by my own runs (`results/bench/*.json`, usage CSVs, and the Spark UI / Ray Dashboard screenshots below).

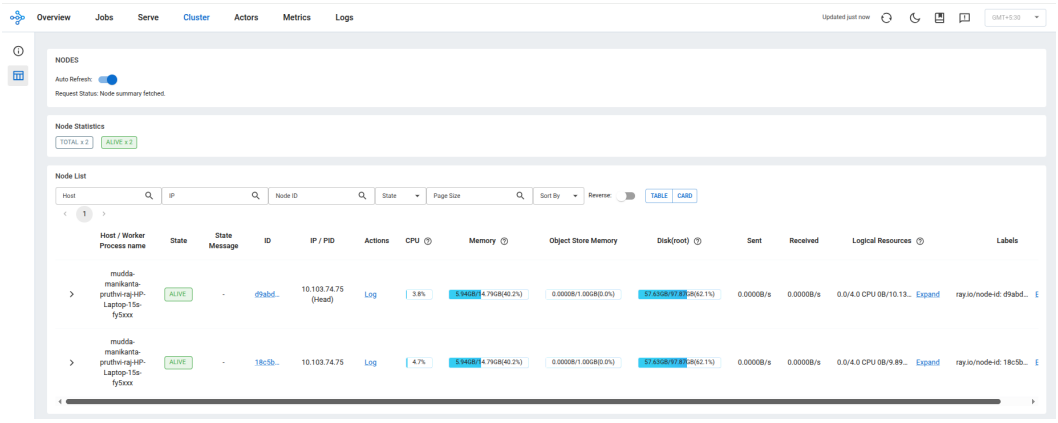
Cluster evidence



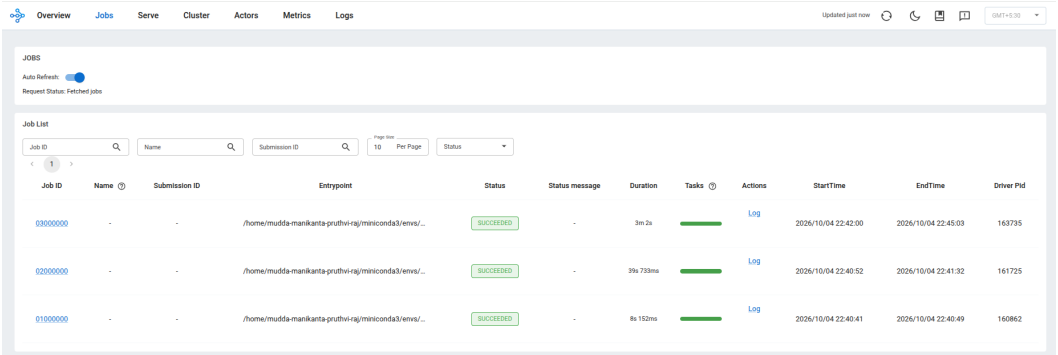
(a) Spark master (port 8080), captured *during* a run: 2 workers ALIVE, 8 of 8 cores in use, application `spark_clean` running.



(b) Spark UI (port 4040), executors page during a run: one executor on each worker, 4 cores each.



(c) Ray Dashboard (port 8265), cluster tab: 2 nodes ALIVE (head and worker), 4 CPUs each. Captured after the runs, so CPU is idle.



(d) Ray Dashboard jobs tab: SUCCEEDED jobs for the 2-file (8 s), 11-file (40 s) and 44-file (3 min 2 s) runs.

Figure 4: Cluster evidence (cropped to the relevant panels).